Simulation Pre-training for Vision-Based End-to-End Lane Keeping: A Scaled-Vehicle Case Study with ROS2/Gazebo

Myungkyun Kim[1)]·Sugyeong Kim[2)]·Eunje Jung[2)]·Soojin Kim[1)]·Hye Hyeon Park[3)]·Jae Wook Jeon[*3)]

[1)] *Department of Electronic and Electrical Engineering, Sungkyunkwan University, 2066, Seobu-ro, Jangan-gu, Suwon-si, Gyeonggi-do 16419, Korea*

[2)] *Department of Mechanical Engineering, Sungkyunkwan University, 2066, Seobu-ro, Jangan-gu, Suwon-si, Gyeonggi-do 16419, Korea*

[3)] *Department of Electrical and Computer Engineering, Sungkyunkwan University, 2066, Seobu-ro, Jangan-gu, Suwon-si, Gyeonggi-do 16419, Korea*

Abstract : This paper examines whether simulation pretraining improves lane keeping when a vision-based end-to-end controller is transferred to a scaled vehicle. The controller maps images from a forward-facing camera directly to steering commands while the speed command remains fixed. We built a single ROS2/Gazebo Classic workflow for simulation, data recording, PilotNet training, and physical deployment, then trained the network with 14,132 simulated images and 5,916 images collected on the physical course. In matched 50-epoch ablations, initializing with the simulation model lowered real-image test MAE by 18.02% at a learning rate of 2 × 10−5 and by 9.61% at 10−4, compared with random initialization on real data. We also evaluated both matched learning-rate pairs on the same indoor physical course, with 30 laps per model at each learning rate. At 10−4, the fine-tuned model completed 28/30 laps (93.3%) without lane departure, compared with 16/30 (53.3%) for the real-only model. At 2 × 10−5, the corresponding results were 27/30 (90%) and 12/30 (40%). The observed completion gaps were 40 and 50 percentage points, respectively. At 2 × 10−5, successful-lap steering metrics differed by 0.3–2.7%, without a consistent smoothness advantage for either model. Both offline and vehicle evaluations therefore favor simulation initialization in the tested configuration. Only one seed and one checkpoint per regime at each learning rate were evaluated on the vehicle, limiting generalization. The evidence is limited to this course and configuration.

Key words : End-to-End Driving, Simulation Pre-training, Lane Keeping, PilotNet, ROS2, Gazebo

1. INTRODUCTION

End-to-End (E2E) driving maps sensor measurements directly to vehicle commands, avoiding some of the intermediate representations required by modular pipelines. Bojarski et al. showed that a convolutional network can learn steering-relevant road features from paired camera images and steering labels.[1)] Collecting enough of these pairs on a physical vehicle is difficult, however, because coverage of uncommon situations requires time and may place the vehicle or nearby people at risk.

Collecting part of the training data in simulation lets us repeat a recording under controlled conditions without another physical drive. Gazebo supplies the virtual course and simulated sensor streams.[2)] We use ROS2 for communication between the recording and control nodes in both environments.[3)] Transferring a trained controller then requires agreement at their interfaces: the images must pass through the same preprocessing, and the steering output must reach the appropriate actuator bridge. Fig. 1 shows how we connected these components to the training code.

The main obstacle is the reality gap. Rendered scenes differ from camera images, simulated sensors omit part of the physical noise, and modeled actuation rarely reproduces the vehicle exactly. Domain randomization addresses part of this mismatch,[4)] while other autonomous-driving studies have adapted images or learned features between domains.[5)] A separate concern is evaluation: low prediction error on recorded frames does not necessarily imply stable closed-loop driving.[6)] We therefore evaluate simulation pretraining once with matched offline runs and again on the vehicle.

Successful sim-to-real deployment does not by itself establish whether simulation pretraining offers an

*Corresponding author, E-mail: jwjeon@skku.edu

advantage over training on real data alone. We examine this question using PilotNet on a scaled vehicle within a ROS2/Gazebo workflow that links data collection, network training, and deployment. At each of two learning rates, simulation-initialized and real-only models use the same physical-course dataset and real-data training conditions for 50 epochs, with initialization as the experimental difference. We evaluate the same selected checkpoints offline and on the physical course, recording lane departures and steering commands. This comparison tests whether lower steering prediction error is accompanied by improved course completion, while assessing steering smoothness as a separate outcome.

2. SIMULATION PLATFORM

2.1 Design and ROS2 Architecture

Seven ROS2 packages handle interfaces, cameras, LiDAR, decision making, actuation, simulation, and launch. Among 23 custom messages, MotionCommand provides the common control interface: one steering command and separate left and right wheel speeds. Rule-based and learned controllers both publish it, so the simulator drive bridge does not change between controllers.

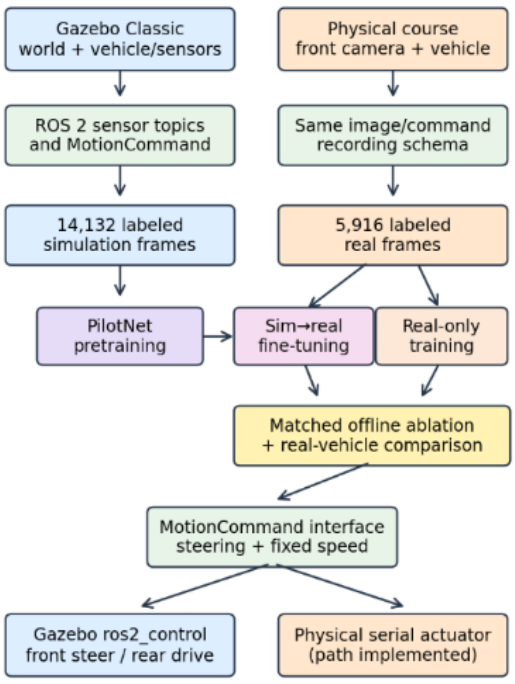


Fig. 1 Recording, training, and deployment through the shared ROS2 command interface.

A saved training sample pairs a JPEG image with a CSV record of its control label (Fig. 1). After training, the controller publishes through the same MotionCommand interface. On the simulator side, ros2_control receives position targets for the front steering joints and velocity targets for the rear wheels. The physical vehicle instead receives commands through a serial-actuation node subscribed to that topic. This node must be started separately: the evaluation launch runs the camera, PilotNet inference, and logger, but a control-mode declaration does not activate the external bridge. Both deployed models use the same input preprocessing and command-publication path, with command and event logging during the trials.

2.2 World, Vehicle, and Sensors

Table 1 Course, sensor, and drive settings in Gazebo

Component	Current configuration
Front RGB camera	1280 × 720, 30 Hz, HFOV 1.0856 rad
Rear RGB camera	720 × 1280, 30 Hz, HFOV 1.0856 rad
2-D LiDAR	360 rays, 10 Hz, 0.12–12.0 m
Drive control	Front position + rear velocity, 100 Hz
Command scaling	steer ±10 → nominal ±0.4394 rad
World surface	16 m × 12 m indoor-course model

For lane-following recordings, the vehicle runs on a 16 m × 12 m course surface within an elliptical indoor scene. Its initial position can vary by up to ±0.05 m and its yaw by ±0.1 rad in the general-driving configuration. A top camera, traffic lights, obstacles, and parking assets are also present for use by other launch configurations. Table 1 lists the implemented settings. We have not yet quantified how closely the geometry, appearance, and vehicle dynamics match their physical counterparts. The world is consequently treated as a course model, without claiming the agreement required of a calibrated digital twin.

The reported network takes its input from /cam0/image_raw. Neither the rear camera nor the planar LiDAR, which has 360 samples per scan, supplies input to this steering policy; those sensors serve the mission and parking configurations. Gaussian noise is specified for images and ranges in the SDF. At the actuator interface, a turn command produces different inner and outer front-wheel angles and rear-wheel speeds, giving Ackermann-like motion. The angle and speed conversion factors are nominal software values. Their correspondence to

physical steering and wheel motion still needs to be measured.

2.3 Task and Data Lifecycle

General driving uses the front camera for lane following and supplies the experiments reported here. The wider workspace also includes a mission configuration with obstacles, a traffic signal, and LiDAR. Parking uses the front and rear cameras, LiDAR, and a six-state controller. Modular perception and control nodes can supply demonstrations for these tasks, while a learned policy can reuse their sensor topics and command interface. We selected general driving for evaluation because simulated and physical image datasets were available together with repeated vehicle trials on the course.

When an image arrives, the recorder pairs it with the latest available command. A missing command, or one older than the configured timeout, causes the image to be rejected. For accepted samples, the CSV retains the image timestamp and save time alongside steering, wheel speeds, and command age. The median age is 0.0266 s for simulated samples and 0.0664 s for physical samples. Recording this delay allows the association between image and label to be inspected later. It does not make the two streams hardware-synchronized.

3. EXPERIMENTAL DESIGN

3.1 Datasets and PilotNet

We collected 14,132 RGB frames in Gazebo and 5,916 on the physical course. All CSV entries point to existing 1280 × 720 JPEGs. Every speed label is 255, leaving only steering to be learned. The physical steering labels have a more negative mean than the simulated labels: −3.467 compared with −2.127. Their respective ranges are −7 to 5 and −6 to 5 command units. Adaptation must therefore be interpreted in light of a label-distribution shift as well as the visual differences in Fig. 2.



Fig. 2 Gazebo and physical-course camera views. The yellow line bounds the lower 65% retained for preprocessing.

Table 2 Image counts after the ordered CSV split

Domain	Total	Train	Val.	Test
Gazebo	14,132	11,306	1,413	1,413
Physical	5,916	4,734	591	591

Windows, columns, furniture, and surface reflections are visible around the physical course in Fig. 2. Gazebo reproduces the related road layout and markings but simplifies much of this surrounding appearance and illumination. For each domain, we assign the first 80% of CSV rows to training, the next 10% to validation, and the final 10% to testing (Table 2). This keeps the stored sequence order. It places the holdout later in the recording without making it independent of the training data by route or acquisition day.

Before a frame reaches PilotNet, we crop away its upper 35%, resize the remainder to 200 × 66, convert it to RGB, and normalize the pixel values to approximately [−1, 1]. Training adds brightness and contrast variation, occasional Gaussian blur, and horizontal flips; a flip also reverses the steering label's sign. The network has five convolutional layers followed by fully connected layers of 100, 50, 10, and one unit. ELU activations and dropout of 0.2 are used with a tanh output. This implementation contains 252,219 trainable parameters.

3.2 Training Comparisons

We use matched training comparisons for offline and real-vehicle evaluation. In the offline ablation, only the network initialization changes within each learning-rate pair. A simulation model is trained for 50 epochs at a learning rate of 10−4, and its best validation checkpoint, from epoch 48, initializes the fine-tuned runs. At each target learning rate, 2 × 10−5 and 10−4, we train the simulation-initialized model and a randomly initialized real-only model for exactly 50 epochs on the physical dataset. Early stopping is disabled, and the checkpoint with the lowest validation loss is retained. Every run uses SmoothL1 loss, AdamW, batch size 64, weight decay 10−5, seed 42, the same split, and the same augmentation. No layer is frozen.

For the physical trials, we deploy both matched learning-rate pairs (10−4 and 2 × 10−5) from the 50-epoch runs described above. The fine-tuned checkpoint is initialized from the 50-epoch simulation model, while the real-only checkpoint is initialized randomly. Within each pair, both models are trained on the same physical dataset for 50 epochs with the same learning rate, optimizer, batch size, weight decay, seed, data split, augmentation, and checkpoint-selection criterion. Early stopping is disabled, and the checkpoint with the lowest validation loss is selected for each model. Thus, the real-data training conditions are matched between the deployed policies.

During inference, we multiply the tanh output by 10, clip it to ±10, and publish the result as an integer steering command. Both policies use the same inference and command-publication settings, with a fixed wheel-speed command of 255. Derivative-based metrics are calculated from the recorded monotonic publication times rather than assuming a fixed interval.

3.3 Offline and Real-Vehicle Evaluation

Offline error is measured before clipping and integer conversion. Let u_i be the labeled steering command and $\hat{u}_i$ the prediction. We compute mean absolute error (MAE) and root mean square error (RMSE) on the last 10% of the physical CSV, which contains 591 ordered frames. The reported values are in project command units, not steering-wheel degrees or road-wheel radians.

$$MAE = \sum|u_i - \hat{u}_i|/N,$$

$$RMSE = \sqrt{[\sum(u_i - \hat{u}_i)^2/N]} \quad (1)$$

Both matched 50-epoch model pairs were evaluated on the same indoor physical course. Each model was driven for 30 laps at each learning rate, giving 120 attempts in total. Trials were conducted in blocks of ten laps, with sufficient cooling time for the battery and switched-mode power supply (SMPS) between blocks. Lane-departure-free completion was the primary outcome.

Lane-departure-free completion rate is the primary endpoint, summarized for each policy. We report it descriptively because repeated laps within a session may be correlated. Secondary measures are calculated from the integer steering command u and its publication time t. They include within-lap command standard deviation, the mean and 95th percentile of absolute command rate, and the RMS and 95th percentile of the absolute second derivative. For interval Δt_i, the rate is $v_i = \Delta u_i/\Delta t_i$, and the second derivative is $a_i = 2(v_i - v_{i-1})/(\Delta t_i + \Delta t_{i-1})$. This second derivative measures command oscillation; it is not physical lateral jerk. Each lap receives equal weight, and summaries are reported as the mean ± between-lap sample standard deviation. Successful-lap results describe only completed runs, whereas all-attempt summaries include the shorter trajectories produced by early failures.

4. RESULTS AND DISCUSSION

4.1 Schedule-Matched Offline Ablation

Table 3 Matched 50-epoch real-image test results

Initialization	LR	MAE	RMSE	MAE change
Simulation	2×10^{-5}	0.373	0.502	−18.02%
Random	2×10^{-5}	0.455	0.593	Reference
Simulation	1×10^{-4}	0.357	0.482	−9.61%
Random	1×10^{-4}	0.395	0.518	Reference

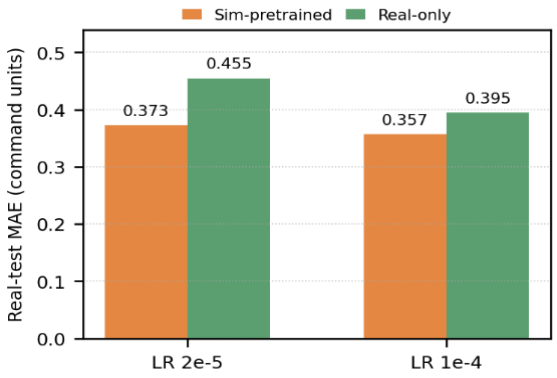


Fig. 3 Schedule-matched real-image MAE by initialization and learning rate.

At 2×10^{-5}, the simulation-initialized run has a real-test MAE of 0.373, compared with 0.455 for random initialization, and an RMSE of 0.502 rather than 0.593. These correspond to reductions of 18.02% and 15.20%. Raising the learning rate to 10^{-4} narrows the MAE reduction to 9.61% (0.395 to 0.357); RMSE falls by 7.04% (0.518 to 0.482). Both comparisons favor simulation initialization within the common 50-epoch real-data budget. Pretraining alone gives a different result: the 50-epoch simulation-only checkpoint records MAE/RMSE of 0.246/0.350 on simulated test images but 1.776/2.131 on physical images. That loss of accuracy across domains provides a reason to adapt the network with real data.

4.2 Real-Vehicle Closed-Loop Results

Table 4 Measured closed-loop completion rates of matched 50-epoch checkpoints

Policy	LR	LDF / N	Departures	Rate
Fine-tuned	1 × 10−4	28/30	2	93.3%
Real-only	1 × 10−4	16/30	14	53.3%
Fine-tuned	2 × 10−5	27/30	3	90%
Real-only	2 × 10−5	12/30	18	40%

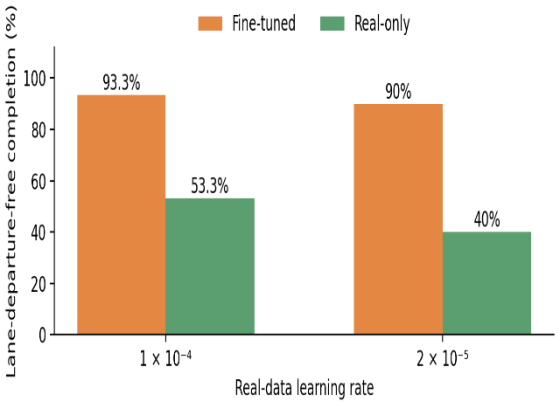


Fig. 4 Measured lane-departure-free completion rates of matched 50-epoch policies.

At a learning rate of 10−4, the fine-tuned policy completed 28 of 30 laps without lane departure (93.3%), with two departures; the real-only policy completed 16 of 30 laps (53.3%), with 14 departures (Table 4). At $2 \times 10-5$, the fine-tuned policy completed 27 of 30 laps (90%), with three departures, while the real-only policy completed 12 of 30 laps (40%), with 18 departures. The observed completion-rate gaps were 40 and 50 percentage points, respectively. Each comparison uses models trained for 50 epochs under the same real-data conditions at the corresponding learning rate.

Table 5 Successful-lap steering metrics at LR $2 \times 10-5$ (mean ± between-lap SD)

Metric (unit)	Fine-tuned N = 27	Real-only N = 12
Command SD (command)	3.005 ± 0.047	3.068 ± 0.061
Mean |rate| (command/s)	1.942 ± 0.078	1.954 ± 0.134
P95 |rate| (command/s)	10.190 ± 0.347	10.221 ± 0.424
RMS 2nd deriv. (command/s²)	61.466 ± 2.211	59.840 ± 3.804
P95 |2nd deriv.| (command/s²)	112.537 ± 4.038	111.353 ± 5.634

Among completed laps at a learning rate of $2 \times 10-5$, the fine-tuned model has 2.05% lower command standard deviation and 0.60% lower mean absolute rate. Its second-derivative RMS, however, is 2.72% higher. The differences in the two P95 measures are also small: the fine-tuned model is 0.30% lower in command rate and 1.06% higher in absolute second derivative. Table 5 therefore does not show a consistent smoothness advantage for either policy. These are descriptive results, not evidence of statistical equivalence, and the successful-lap filter selects different subsets from the two policies.

4.3 Interpretation and Validity

The offline and vehicle experiments provide complementary evidence. Under matched schedules, simulation initialization reduces prediction error on held-out physical images. On the course, the matched fine-tuned policy also achieves a higher lane-departure-free completion rate than the real-only policy at both learning rates. The holdout frames are not the same inputs encountered during closed-loop driving. Previous work likewise found that agreement between model-level and system-level evaluation depends on the selected images, labels, and pre-failure intervals.6) The two evaluation levels therefore provide different information about controller behavior. Prior vehicle-control research has also combined simulation and physical experiments when evaluating path-following methods.7)

Our claim is restricted to the tested configuration. Within each learning-rate pair, the deployed policies share the same real-data optimization schedule, but only one seed and one checkpoint per regime at each learning rate were driven on one course. Repeated laps may be correlated, and battery state, surface condition, illumination, and execution order remain potential

sources of variation. We also lack localization-based cross-track error. The frame split is ordered rather than independent by session, and the simulator has not been fully calibrated for geometry, appearance, or actuation. A stronger follow-up study should evaluate more seeds and courses, randomize policy order, define lane and finish criteria in advance, and measure lateral error automatically.

5. CONCLUSION

We implemented a ROS2/Gazebo Classic workflow that links the indoor course, vehicle and sensor models, a reusable command interface, data recording, PilotNet training, and deployment on the physical platform. The same forward-camera steering task can therefore be studied through a controlled offline comparison and through synchronized closed-loop vehicle logs.

Using 14,132 simulated images and 5,916 physical-course images, simulation initialization reduces real-test MAE by 9.61–18.02% in the matched 50-epoch experiments. In the vehicle evaluation of models trained under matched real-data conditions for 50 epochs, the fine-tuned checkpoint completes 28/30 laps (93.3%) without lane departure at 10−4, compared with 16/30 (53.3%) for the real-only checkpoint. At 2 × 10−5, the results are 27/30 (90%) and 12/30 (40%), respectively. The completion-rate gains are 40 and 50 percentage points, respectively. At 2 × 10−5, successful-lap steering metrics are close and point in mixed directions, showing no consistent smoothness advantage.

The findings support simulation pretraining within this workflow under matched real-data training conditions. Generalization remains limited because only one seed and one checkpoint represent each regime at each learning rate and the vehicle evaluation covers one course. Future experiments should repeat training with additional seeds, vary the course, randomize policy order, and add automatic lateral-error measurements.

Acknowledgements

This work was supported by the Korea Institute for Advancement of Technology (KIAT) grant funded by the Ministry of Education (Project No. P0030064, Advanced Industry Talent Training Bootcamp Program – Sungkyunkwan University (Future Mobility)).

References

1) M. Bojarski, D. Del Testa, D. Dworakowski, B. Firner, B. Flepp, P. Goyal, L. D. Jackel, M. Monfort, U. Muller, J. Zhang, X. Zhang, J. Zhao, and K. Zieba, "End to End Learning for Self-Driving Cars," arXiv:1604.07316, 2016.

2) N. Koenig and A. Howard, "Design and Use Paradigms for Gazebo, an Open-Source Multi-Robot Simulator," Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems, Vol. 3, pp. 2149–2154, 2004, doi:10.1109/IROS.2004.1389727.

3) S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, Architecture, and Uses in the Wild," Science Robotics, Vol. 7, No. 66, eabm6074, 2022, doi:10.1126/scirobotics.abm6074.

4) J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World," Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems, pp. 23–30, 2017, doi:10.1109/IROS.2017.8202133.

5) C. Hu, S. Hudson, M. Ethier, M. Al-Sharman, D. Rayside, and W. Melek, "Sim-to-Real Domain Adaptation for Lane Detection and Classification in Autonomous Driving," arXiv:2202.07133, 2022.

6) A. Stocco, B. Pulfer, and P. Tonella, "Model vs System Level Testing of Autonomous Driving Systems: A Replication and Extension Study," Empirical Software Engineering, Vol. 28, Article 73, 2023, doi:10.1007/s10664-023-10306-x.

7) A. Kovács and I. Vajk, "Internal Model-Based Robust Path-Following Control for Autonomous Vehicles," International Journal of Automotive Technology, Vol. 25, pp. 249–260, 2024, doi:10.1007/s12239-024-00003-z.